

AN1.6 - Server-side Agent Runtime (A4)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A4 - Server-side resumable Agent Runtime + LimitEnforcer + RunTracer **Depends on:** AF-v1 contracts (A1), Tool Registry + Gateway (A2), Run persistence applied (A3 / G03) **Gate:** G04 - the runtime substrate. **Not** the eight-agent system.

First real runtime milestone. The database (A3 tables) is the durable execution ledger; the runtime is a resumable state machine over it. A lost serverless invocation resumes from the AgentRun row alone.

1. What was built - services/agent-framework/runtime/

Server-only. No import path reaches the browser bundle (asserted).

File	Role
agent-run.repository.ts	The only module that touches the A3 tables. Exposes createRun, getRun, getRunTrace, count*, nextStepIndex, append{Step, ToolCall, Evidence} (append-only - no updateStep/deleteStep), and a narrow patchRun for AgentRun's status + terminal fields + resumeState.
limit-enforcer.ts	Pure checkLimits(ctx, nextCreditEstimate) -> maps each breach to its terminal AgentRunStatus via AF-v1 LIMIT_BREACH_STATUS. Reads the run's snapshot (AgentRun.limits), never the live agent policy.
run-tracer.ts	RunTracer - appends an immutable AgentStep at the next index and mirrors it to logger.child("agent-runtime") with a runId. The audit trail is the rows; the log is an aid.
planner.ts	planRun(definition, input) - the minimal deterministic planner: walks the agent's bound tools in order, shapes the run input per tool, emits PlannerToolRequest[] intentents only (no handler, no I/O). The real LLM-assisted / supervisor-coordinated planner is A5.
authorizer.ts	authorizeToolRequest(def, request, registry, creditsReserved) - the authorization boundary: tool exists -> tool active -> evaluatePermission -> canRunAtAutonomy -> produces an AuthorizedToolIntent or a typed denial.
agent-runtime.ts	AgentRuntime.startRun() / tick() / runToCompletion() - the resumable state machine.
index.ts	server-only barrel.

1.1 Contract note

No new contract change in A4. AgentRuntime accepts an in-memory AgentDefinition and stashes it in AgentRun.metadata.definition at startRun, so tick(runId) is fully self-contained from the DB. A dedicated AgentVersion table (the contract's AgentVersionSnapshot) is a later step.

2. The tick() state machine

```
startRun({ definition, input, userId })
|-- validateAgentDefinition (registry-checked)
```

```

|-- snapshot limits (DEFAULT_RUN_LIMITS, tightened by creditPolicy.perRunCeiling - never loosened)
|-- pre-estimate credits (sum of bound tools' flat costs)
\-- createRun -> status "queued" [NO execution]

tick(runId) -- ONE bounded slice, then persist + return --
1. load run; if terminal -> return (idempotent)
2. read AgentDefinition from run.metadata
3. LimitEnforcer: wall-clock / steps / tool-calls / retries
   breach -> patchRun(status + errorCode) ; return
4. dispatch on run.status:
   queued / planning -> plan:
       append "plan" step (output = the PlannerToolRequest[])
       patchRun { status: running, plan, resumeState: { nextPlanIndex: 0 } }
       return
   running -> one slice:
       nextPlanIndex >= plan.length ?
       YES -> append "output" step ; patchRun { status: succeeded, output, completedAt, resumeState: null }
       NO -> request = plan[nextPlanIndex]
           -- credit pre-check (LimitEnforcer with nextCreditEstimate) -- BEFORE the executor
           breach -> terminal credit_limit ; return
           -- authorize(request) --
           denied -> append tool_call step (error) + AgentToolCall row (denied status)
                   patchRun terminal (tool_error | permission_denied) ; return
           -- invokeTool(AuthorizedToolIntent) -- the ToolGateway
           append "tool_call" step + AgentToolCall row + AgentEvidence rows (linked to the tool call)
           ToolResult.status != ok -> patchRun terminal (tool_error | permission_denied) ; return
           patchRun { creditsConsumed += , resumeState: { nextPlanIndex + 1 } }
           return
   awaiting_approval -> no-op (approval API is a later step)

runToCompletion(runId, { maxTicks }) -- synchronous driver: loops tick() until terminal --

```

Every tick() persists before it returns. Between ticks the only state is the AgentRun row (status, plan, resumeState, creditsConsumed) plus the append-only AgentStep / AgentToolCall / AgentEvidence history.

3. Owner A4 requirements - how each is met

Requirement	Implementation
Server-only execution	services/agent-framework/runtime/* imports only server modules (@/lib/prisma, @/services/*, contract types). Asserted: no next/navigation, no "use client", no browser globals.
Resumable tick(), no long-lived worker	tick() executes exactly one slice (plan / one tool call / output) and returns. runToCompletion() is a convenience loop for the sync case; a scheduler calls tick() across invocations. Proven: a fresh AgentRuntime instance per tick drives a multi-step run to succeeded from the DB row alone.
LimitEnforcer before the executor	checkLimits() runs at the top of every tick() (wall-clock/steps/tool-calls/retries) and again with the next tool's credit estimate immediately before authorizeToolRequest + invokeTool . The credit_limit and step_limit proofs terminate with 0 tool calls .
Enforce the A1 limits	maxSteps -> step_limit, maxToolCalls -> tool_call_limit, maxRuntimeMs -> timeout, maxCreditCost -> credit_limit, maxRetries -> failed (AF-v1 LIMIT_BREACH_STATUS). Snapshot taken at startRun; a mid-run policy edit cannot change a running run's ceilings.

Requirement	Implementation
RunTracer - durable, not console.log	Every transition writes an AgentStep row (plan, tool_call, evidence, output) with startedAt/completedAt/durationMs. logger.info is emitted alongside with runId/stepId correlation but is never the source of truth.
Planner != Executor != Registry	planner.ts emits PlannerToolRequest intents; authorizer.ts converts to AuthorizedToolIntent only after all checks; invokeTool() (A2 gateway) is the only executor. The planner holds no handler.
A4 proves the substrate, not eight agents	One minimal deterministic planner. No supervisor, no LLM, no agent templates, no /dashboard/agents change, no API routes.

4. G04 proof - npm run validate:agent-runtime -> 8 passed, 0 failed

Writes real rows to the A3 tables under a synthetic userId, cleans up at start + end.

4.1 Small end-to-end proof (the deliberately small one)

```
Agent (MARKET_INTELLIGENCE, tools:[market.snapshot], perm:[CAN_READ_MARKET_DATA], autonomy 1)
-> startRun { input: { symbol: "XAUUSD" } }          -> AgentRun status "queued"
-> runToCompletion:
  tick: queued -> planning -> running (AgentStep #0 kind "plan")
  tick: authorize(market.snapshot) -> invokeTool -> MarketDataService [REAL]
        AgentStep #1 "tool_call" (ok) + AgentToolCall (ok) + AgentEvidence #1 (market_data)
        AgentStep #2 "evidence"
  tick: plan exhausted -> AgentStep #3 "output" -> status "succeeded"
```

Asserted: status succeeded; steps [plan, tool_call, evidence, output] with **gapless indices 0..3**; exactly 1 AgentToolCall (toolId=market.snapshot, status=ok); >=1 AgentEvidence with provenance.producer="market-data-service", runId set, toolCallId linked; run.output.evidenceCount matches; a second tick() on the terminal run is a **no-op** (idempotent).

Live result this run: succeeded, 4 steps, 1 evidence (XAUUSD via twelve-data). If the provider were down the run would terminate tool_error with the trace still intact - the test accepts either and validates persistence both ways.

4.2 Resumable proof

Fake dependency-free 2-tool agent, a **fresh AgentRuntime instance per tick()** (simulating lost serverless invocations):

after	status	tool calls	resumeState.nextPlanIndex
tick 1	running	0	0 (plan persisted, len 2)
tick 2	running	1 (test.step_a)	1
tick 3	running	2 (test.step_b)	2
tick 4	succeeded	2	resumeState = null

Asserted: final steps [plan, tool_call, evidence, tool_call, evidence, output], gapless indices across ticks; 2 evidence rows; creditsConsumed = 2 (placeholder 1+1 accumulated across ticks); completedAt set.

4.3 Failure / limit paths (deterministic, no network)

Scenario	Terminal status	errorCode	tool calls
maxSteps: 1	step_limit	maxSteps	0
credit ceiling 0.5 < first tool cost 1	credit_limit	maxCreditCost	0 (pre-executor)
bound tool does .not .exist	tool_error	unknown_tool	1 (invalid_input)
agent lacks CAN_READ_MARKET_DATA	permission_denied	missing_permission	1 (permission_denied, pre-executor)
autonomy 1 vs tool autonomyFloor 2	permission_denied	autonomy_floor	1

4.4 Forensic query

SELECT * FROM "AgentEvidence" WHERE "runId" = ? - every row has runId, stepId, and a provenance.producer. "What did the agent do, which tool, what evidence, what conclusion" is answered from getRunTrace(runId) (run + ordered steps + tool calls + evidence) with **no log replay**.

4.5 Regression

validate:agent-contracts 38/0 - validate:agent-tools 20/0 - validate:agent-run-persistence 17/0.

5. TypeScript

npx tsc --noEmit: **0 errors** in services/agent-framework/**, types/agent-framework/**, scripts/validate-agent-*.ts. Repo-wide 77, all in the stale generated .next/dev/types/validator.ts (gitignored, pre-existing).

6. Files changed

Added (8):

```
frontend/services/agent-framework/runtime/agent-run.repository.ts
frontend/services/agent-framework/runtime/limit-enforcer.ts
frontend/services/agent-framework/runtime/run-tracer.ts
frontend/services/agent-framework/runtime/planner.ts
frontend/services/agent-framework/runtime/authorizer.ts
frontend/services/agent-framework/runtime/agent-runtime.ts
frontend/services/agent-framework/runtime/index.ts
frontend/scripts/validate-agent-runtime.ts
```

Modified (1):

```
frontend/package.json    + "validate:agent-runtime"
```

Not touched: Prisma schema, contracts, tool layer, legacy services/agents/* + /dashboard/agents, every other route/service. No new API routes.

7. Explicit non-goals honoured

- No supervisor / orchestrator (A5) - no LLM anywhere in the runtime
- No agent templates, no eight agents - no /dashboard/agents change
- No API routes (the runtime is a service; HTTP wiring comes with the first real agent)

- No `awaiting_approval` flow (no L3 tools in v1 - `tick()` is a documented no-op there)
 - No real credit ledger (A9) - the gateway's placeholder pre-estimate is stamped and accumulated; hard enforcement of a *user* balance is A9. Per-run `maxCreditCost` **is** enforced now.
-

8. Status

A4 complete. The runtime substrate works end to end against real services and persists a full forensic trace. Next: **A5 - Supervisor / Orchestrator** (the real planner: goal decomposition, specialist coordination, LLM-assisted planning behind the `lib/ai` seam), then A6+ per AN1.1 sec 7.

G04 review requested.

End AN1.6.